

Data Lineage Estático con IA: Reconstrucción Automática y Continua del Linaje de Datos a Nivel Columna mediante Análisis de Código Fuente

Shanon Samora

Profesora: Ana Darcacha – Universidad de Palermo

Resumen-- En las organizaciones data-intensive, los datos atraviesan decenas de transformaciones antes de llegar a reportes y dashboards. Con el tiempo, este flujo se vuelve tan complejo que nadie puede responder dónde originan los números críticos. El Data Lineage resuelve esto rastreando el origen de cada dato a nivel columna. Este trabajo propone un sistema de análisis estático de código fuente que reconstruye automáticamente el linaje de datos y lo mantiene actualizado de forma continua a través de integración con repositorios Git. El enfoque híbrido combina un parser determinístico (sqlglot) para SQL estándar con un modelo de lenguaje (LLM) que interpreta casos complejos: queries dinámicas, transformaciones en pandas/PySpark y procedimientos almacenados. El resultado se almacena en una base de datos de grafos (Neo4j) y se expone a través de una interfaz web y API REST. La validación demuestra que el enfoque híbrido mejora significativamente la cobertura y precisión del lineage respecto al parsing determinístico solo, resolviendo el vacío tecnológico entre herramientas comerciales inaccesibles y soluciones open-source incompletas.

Palabras clave--: data lineage, análisis estático de código, inteligencia artificial, cumplimiento regulatorio, trazabilidad de datos, bases de datos de grafos, SQL, Python, BCBS 239, SOX.

I. INTRODUCCIÓN

En las organizaciones modernas, los datos son un activo crítico. Sin embargo, comprender el origen de un dato específico — por ejemplo, cómo se calcula un KPI regulatorio — se ha convertido en una tarea casi imposible. Los datos pasan por docenas de transformaciones en múltiples etapas: se extraen desde fuentes heterogéneas, se limpian en procesos intermedios, se combinan con otros datasets, se reprocesan con lógica de negocio compleja y finalmente llegan a un reporte o dashboard. A medida que los pipelines de datos crecen en volumen y complejidad, surge una pregunta fundamental que nadie en la organización puede responder con certeza: ¿de dónde proviene realmente este número?

El concepto de Data Lineage (linaje de datos) apunta a resolver exactamente este problema. Se define como la capacidad de rastrear la procedencia de un dato, identificar todos los pasos de transformación que sufrió a lo largo de su ciclo de vida y visualizar la cadena completa de origen hasta su presentación

final. En la industria financiera, esta trazabilidad no es una conveniencia técnica: es una obligación regulatoria. Normativas como BCBS 239 (principios de agregación de datos de riesgo del Comité de Basilea), SOX (Sarbanes-Oxley Act) y MiFID II (Markets in Financial Instruments Directive) exigen explícitamente que las instituciones financieras demuestren la procedencia de los datos utilizados en reportes regulatorios. El incumplimiento genera multas de millones de dólares, sanciones administrativas e incluso pérdida de licencia operativa.

Hoy, las organizaciones resuelven esta obligación de dos modos insatisfactorios. El primero es documentación manual: los equipos crean spreadsheets o wikis que describen dónde originan los datos y cómo se transforman. El problema es que esta documentación inevitablemente se desactualiza: cada cambio en el código requiere actualizar manualmente la documentación, lo que nunca ocurre consistentemente. El segundo modo es adquirir herramientas comerciales complejas — Collibra, Atlan, Informatica — que automaticen la trazabilidad. Estas soluciones funcionan bien pero cuestan entre USD 100.000 y USD 500.000 anuales, disponible solo para grandes empresas con presupuestos de TI significativos.

Entre estos dos extremos existe un vacío tecnológico que este proyecto propone ocupar.

II. JUSTIFICACIÓN

Tres factores hacen que este proyecto sea relevante y oportuno:

A. Obligación regulatoria sin solución accesible

Regulaciones como BCBS 239, SOX y MiFID II exigen que las instituciones financieras demuestren la procedencia de los datos utilizados en reportes regulatorios. El incumplimiento genera multas de millones de dólares, sanciones administrativas e incluso pérdida de licencia operativa. Hoy, las organizaciones resuelven esta obligación con documentación manual — proceso lento, propenso a errores y que se desactualiza con cada cambio de código. Las herramientas comerciales que automaticen la trazabilidad a nivel columna (Collibra, Atlan, Informatica, Governance Catalog) tienen costos entre USD

100.000 y USD 500.000 anuales, disponible solo para grandes empresas. Una solución autoalojable y de bajo costo resolvería una necesidad urgente que millones de equipos de datos en el mundo enfrentan hoy.

B. Vacío tecnológico concreto

Existe una brecha entre las herramientas open-source (OpenLineage, Marquez, dbt) que solo cubren lineage en tiempo de ejecución, y las comerciales que cubren lineage estático pero a costos prohibitivos. El problema es que el código legacy — scripts SQL, notebooks de Jupyter, transformaciones en Python con queries dinámicas, procedimientos almacenados — representa la mayoría del código real de una organización, pero queda completamente invisible para las herramientas de lineage runtime. Ninguna solución open-source combina parsing determinístico con interpretación por IA para extraer linaje de código fuente sin ejecución. Este proyecto apunta a cubrir ese espacio intermedio.

C. Valor de negocio tangible más allá del compliance

El linaje continuo habilita tres capacidades inmediatas por las cuales una organización invertiría: (a) análisis de impacto automatizado — antes de modificar una columna en una tabla fuente, saber instantáneamente qué reportes, dashboards y pipelines downstream se ven afectados, reduciendo incidentes por cambios no evaluados; (b) reducción del tiempo de debugging — cuando un KPI muestra un valor incorrecto, trazar la cadena completa de origen en segundos en lugar de horas de investigación manual; y (c) documentación siempre actualizada — el linaje se regenera con cada cambio de código, eliminando la necesidad de mantener documentación manual que inevitablemente se desactualiza.

III. MARCO TEÓRICO

A. Data lineage y gestión de metadatos

El data lineage es la trazabilidad del recorrido que sigue un dato desde su origen hasta su uso final, incluyendo los procesos que lo crean, lo transforman y lo consumen. En entornos de gobierno de datos, esta trazabilidad se considera parte de la gestión de metadatos, porque los metadatos describen qué es cada activo de información, de dónde proviene y cómo debe interpretarse. En este trabajo, el foco está en el lineage a nivel columna, ya que ese nivel de granularidad permite identificar dependencias precisas entre campos, evaluar el impacto de cambios y respaldar auditorías con mayor exactitud [1][2].

B. Análisis estático de código fuente

El análisis estático de código es una técnica que examina el código fuente de un programa sin ejecutarlo, con el objetivo de extraer información sobre su estructura, dependencias o comportamiento (ej: puntos de transformación en SQL, Python o notebooks). Es especialmente valioso para extraer linaje de código legacy o de scripts que no pueden ejecutarse en entornos controlados de análisis, cubriendo así la mayoría del código real que existe en una organización. A diferencia del análisis en tiempo de ejecución (o runtime), que requiere correr el

programa para observar su ejecución real, el análisis estático opera directamente sobre el código. Su principal ventaja es la escalabilidad para auditar repositorios completos; su principal desafío es resolver construcciones dinámicas y ambigüedades contextuales [3].

C. Granularidad de lineage: nivel tabla y nivel columna

La granularidad del lineage puede ser a nivel tabla o a nivel columna. El nivel tabla identifica dependencias generales entre datasets, pero no explica cómo se calcula un campo específico. El nivel columna vincula cada atributo de salida con sus atributos de entrada y con la lógica aplicada, lo que habilita explicabilidad fina en métricas regulatorias, debugging de KPIs y validación técnica de reportes [4].

D. Lineage en tiempo de ejecución

El lineage runtime se obtiene observando ejecuciones de pipelines y jobs, por lo que captura con precisión lo que efectivamente corrió. El lineage estático se reconstruye analizando código fuente y artefactos de transformación sin ejecutar procesos. Ambos enfoques son complementarios: el runtime ofrece evidencia operacional, mientras el estático amplía cobertura sobre código no instrumentado y permite análisis preventivo antes del despliegue [4].

E. Cumplimiento regulatorio en datos financieros

En el sector financiero, la trazabilidad no es solo una buena práctica técnica, sino una exigencia normativa. BCBS 239 establece principios de agregación y reporte de datos de riesgo con foco en exactitud, integridad y trazabilidad. SOX exige controles internos robustos sobre información que impacta reportes corporativos. MiFID II impone obligaciones de registro y transparencia sobre datos vinculados a mercados e instrumentos financieros. En conjunto, estas normas justifican soluciones de lineage verificable y auditable como capacidad estructural de cumplimiento [5][6][7].

F. Representación del lineage en bases de datos orientadas a grafos

Una base de datos de grafos modela entidades como nodos y relaciones como aristas, esquema adecuado para representar dependencia entre tablas, columnas, procesos y artefactos de código. En ese modelo, el lineage se consulta mediante recorridos, ancestros, descendientes e impacto transitive, operaciones que suelen ser costosas en enfoques puramente relacionales. Por eso los grafos ofrecen una base natural para visualización explicable y análisis de impacto en tiempo de consulta [8].

G. Herramientas de data lineage y gobierno de metadatos

OpenLineage es un estándar abierto para registrar metadatos de ejecución en pipelines de datos. Define un modelo común de eventos (job, dataset de entrada y salida, estado de ejecución) para que distintas herramientas puedan intercambiar información de lineage runtime de forma interoperable [9]. Marquez es una plataforma open-source que implementa OpenLineage para almacenar y consultar esos eventos. Permite visualizar la traza de ejecución entre jobs y datasets, y funciona

como backend de metadatos para observabilidad y auditoría técnica [10].

dbt es una herramienta de transformación analítica en SQL que organiza modelos en un DAG de dependencias. Ese grafo permite entender qué modelos dependen de cuáles y exponer lineage dentro del proyecto dbt, especialmente en entornos de analytics engineering [11].

Collibra es una plataforma empresarial de gobierno de datos que integra catálogo, glosario, políticas y lineage. Su enfoque está orientado a cumplimiento, trazabilidad organizacional y análisis de impacto sobre activos de datos críticos [12].

Atlan es una plataforma de catálogo y gobernanza basada en metadatos activos. Centraliza información técnica y de negocio, y ofrece lineage para facilitar colaboración entre equipos de datos, documentación y evaluación de impacto ante cambios [13].

Informatica ofrece capacidades de lineage dentro de su ecosistema de integración, calidad y gobierno de datos. Su propuesta apunta a trazabilidad de extremo a extremo entre fuentes, transformaciones y consumos analíticos en contextos enterprise [14].

Governance Catalog refiere al componente de catálogo de gobierno en plataformas de data governance. Su función es unificar metadatos, clasificación y relaciones de dependencia para que el lineage sea útil no solo para perfiles técnicos, sino también para auditoría, riesgo y cumplimiento (por ejemplo, en Microsoft Purview) [15].

IV. ESTADO DEL ARTE

El Data Lineage se consolidó como una capacidad central en arquitecturas modernas de datos, ya que permite identificar el origen, las transformaciones y el consumo de la información a lo largo de pipelines analíticos y regulatorios. En términos prácticos, su valor no se limita a la documentación técnica: también habilita análisis de impacto, auditoría, cumplimiento normativo y resolución más rápida de incidentes en entornos donde los datos se transforman en múltiples etapas antes de llegar a reportes o dashboards.

Los enfoques existentes pueden agruparse, en primer lugar, en soluciones de observación en runtime. Herramientas como OpenLineage, Marquez y distintos conectores de orquestación capturan eventos durante la ejecución de procesos, lo que permite obtener trazabilidad operativa en tiempo real. Sin embargo, su cobertura depende de que los pipelines estén instrumentados. Esto deja fuera una parte importante del ecosistema real de una organización: scripts legacy, notebooks ejecutados manualmente, procesos ad hoc y transformaciones dispersas en Python o PySpark.

Una segunda familia está compuesta por plataformas de gobierno y catálogo de datos, como Collibra, Atlan, Informatica o Microsoft Purview. Estas soluciones integran metadatos, catálogo y visualización de lineage, y suelen ofrecer capacidades útiles para trazabilidad e impacto de cambios. No obstante, su adopción suele verse limitada por el costo, la complejidad de implementación y la dependencia de ecosistemas propietarios, lo que reduce su accesibilidad para equipos medianos o proyectos que requieren una alternativa autoalojable.

En paralelo, existen propuestas de lineage estático basadas en parsers SQL y análisis de dependencias del código fuente. Este enfoque mejora la cobertura sobre repositorios sin necesidad de ejecutar pipelines, pero presenta límites cuando aparecen SQL dinámico, macros, procedimientos almacenados poco documentados o transformaciones escritas en Python/PySpark. En esos casos, la extracción del linaje a nivel columna pierde precisión justamente en los escenarios que más valor tienen para auditoría y análisis de impacto.

Más recientemente, comenzaron a explorarse modelos de lenguaje (LLM) para comprensión de código, extracción semántica y asistencia en documentación técnica. Estos avances abren la posibilidad de interpretar patrones que los parsers determinísticos no resuelven con facilidad, especialmente en lógica compleja o parcialmente implícita. En paralelo, las bases de datos orientadas a grafos —como Neo4j— se consolidaron como una representación adecuada para modelar relaciones entre tablas, columnas, transformaciones y activos de datos, facilitando consultas de recorrido e impacto con mayor expresividad.

A partir de este relevamiento, se identifica un vacío concreto: no se encontró una solución accesible que combine de forma robusta análisis estático multi-lenguaje, lineage a nivel columna, cobertura de casos no instrumentados y actualización continua integrada al flujo de desarrollo. En ese espacio se posiciona este trabajo, proponiendo un enfoque híbrido que combina parsing determinístico e interpretación asistida por IA para mejorar la cobertura y la precisión sin depender de la ejecución en producción.

V. DESARROLLO

La solución propuesta se diseña como una aplicación web conectada al ciclo de desarrollo de software, con capacidad para analizar repositorios de código y reconstruir el linaje de datos de forma continua. A diferencia de una herramienta que se ejecuta una sola vez sobre un proyecto cerrado, este sistema se integra con GitHub para revisar cambios en cada pull request, actualizar el grafo de lineage y exponer los impactos potenciales antes de que el código llegue a producción. De este modo, el linaje deja de ser un artefacto estático y pasa a funcionar como una capacidad operativa del proceso de desarrollo.

A. Arquitectura general

La arquitectura se organiza en tres capas principales: ingesta, análisis y visualización. La capa de ingesta se encarga de obtener el contenido del repositorio GitHub, identificar archivos relevantes y detectar cuándo ocurre un cambio sobre una rama o un pull request. La capa de análisis procesa el código fuente para extraer relaciones de dependencia entre tablas, columnas y transformaciones. Finalmente, la capa de visualización presenta el lineage resultante a través de una interfaz web y una API que permite consultas programáticas por parte de otros sistemas.

VI. REFERENCIAS

- [1] DAMA International, The DAMA Guide to the Data Management Body of Knowledge (DAMA-DMBOK), 2ª

- ed. Available: <https://books.google.com.co/books?id=7FdNCwAAQBAJ&printsec=frontcover>
- [2] IBM, “Data Lineage”. Available: <https://www.ibm.com/products/watsonx-data-intelligence/data-lineage>
 - [3] F. Nielson, H. R. Nielson, and C. Hankin, Principles of Program Analysis. Springer, 1999. Available: https://www.researchgate.net/publication/265352570_Principles_of_Program_Analysis
 - [4] R. Kimball and M. Ross, The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling, 3rd ed. Wiley, 2013. Available: <https://ia801609.us.archive.org/14/items/the-data-warehouse-toolkit-kimball/The%20Data%20Warehouse%20Toolkit%20-%20Kimball.pdf>
 - [5] Basel Committee on Banking Supervision, Principles for Effective Risk Data Aggregation and Risk Reporting (BCBS 239). Bank for International Settlements, 2013. Available at: <https://www.bis.org/publ/bcbs239.pdf>
 - [6] U.S. Congress, Sarbanes–Oxley Act of 2002, Public Law 107–204, 2002. Available at: https://www.dol.gov/sites/dolgov/files/oalj/PUBLIC/WHI_STLEBLOWER/REFERENCES/STATUTES/SARBANE_S_OXLEY_ACT_OF_2002.PDF
 - [7] European Parliament and Council, Directive 2014/65/EU on Markets in Financial Instruments (MiFID II), 2014. Available at: <https://eur-lex.europa.eu/eli/dir/2014/65/oj/eng>
 - [8] I. Robinson, J. Webber, and E. Eifrem, Graph Databases, 2nd ed. O’Reilly Media, 2015. Available at: https://web4.ensiie.fr/~stefania.dumbrava/OReilly_Graph_Databases.pdf
 - [9] OpenLineage, “OpenLineage Documentation.” Available: <https://openlineage.io/docs/>
 - [10] Marquez, “Marquez Documentation.” Available: <https://marquezproject.ai/docs/>
 - [11] dbt Labs, “dbt Documentation.” Available: <https://docs.getdbt.com/>
 - [12] Collibra, “Data Lineage.” Available: <https://www.collibra.com/us/en/products/data-lineage>
 - [13] Atlan, “Data Lineage.” Available: <https://atlan.com/data-lineage/>
 - [14] Informatica, “Data Lineage.” Available: <https://www.informatica.com/services-and-training/glossary-of-terms/data-lineage-definition.html>
 - [15] Microsoft, “Microsoft Purview Documentation.” Available: <https://learn.microsoft.com/en-us/purview/>